

The Hash Function CHIME

April 2026

Contents

1	Description of CHIME	3
1.1	Conventions	3
1.2	Notation	3
1.3	Basic Operations	3
1.4	State and State Array Conversion	3
1.5	The CHIME Hash Function	4
1.5.1	CHIME-512 Description	4
1.5.2	CHIME-1024 Description	5
1.6	Round Function	5
1.6.1	Linear Operation λ	5
1.6.2	AddRoundConstant (AC)	6
1.6.3	Non-linear Operation χ	6
1.6.4	Linear Operation τ	7
2	Performance	8
2.1	Software Performance	8
2.2	Hardware Performance	8
3	Choice of Parameters for CHIME	9
3.1	Choice of the Shift and rotation offsets in Round Function	9
3.2	Choice of the Round Constants	9
3.3	Choice of the Number of Rounds	9
4	Security Claims	10
5	Security Analysis	11
5.1	Differential Distribution Table	11
5.2	Collision Attacks	11
5.2.1	Collision Attacks on CHIME-512	11
5.2.2	Collision Attacks on CHIME-1024	11
5.3	Preimage Attacks	12
5.3.1	State Bit Encoding	12
5.3.2	Complexity Calculation for Preimage Attacks	12
5.3.3	Preimage Attack Results	13
5.4	Second-Preimage Attack	16
5.5	Quantum Attack	16
5.6	Resistance to Length Extension Attacks	16
5.7	Linear Analysis	16
5.8	Randomness Testing of CHIME	17
6	Feature Claims	18
A	Appendix: Randomness Tests of CHIME	20
A.0.1	Random Message Mode Testing	21
A.0.2	Output Feedback Mode Testing	21
A.0.3	Low Density Mode Testing	22
A.0.4	High Density Mode Testing	23

1 Description of CHIME

1.1 Conventions

- **Bit string:** A binary sequence consisting of 0s and 1s.
- **Big-endian:** A data representation format in memory where the most significant byte is on the left (lowest address) and the least significant byte is on the right (highest address).
- **Message:** A bit string of arbitrary length. In this document, the message serves as the input data for the hash function.
- **Hash value:** Length of bit string output after the hash function is applied to the message.
- **State:** The bit string updated during the execution of the hash function.
- **State array:** An array representing the state. The hash function CHIME utilizes a 1536-bit state, divided into six 256-bit state arrays.

1.2 Notation

- M : Message.
- $|M|$: The length in bits of a message M .
- M' : Padded message.
- H : Hash value.
- S : State. It contains six 256-bit state arrays $A_0, A_1, B_0, B_1, C_0, C_1$.
- P : Permutation.
- $RC[i]$: The round constant for the i -th round, containing four identical 64-bit words X_i .

1.3 Basic Operations

- $X \parallel Y$: Concatenation of bit strings X and Y .
- $X[i]$: The i -th bit of bit string X , indexed starting from 0.
- $X \oplus Y$: Bitwise XOR of bit strings X and Y of the same length.
- $X \ll n$: left-shift by n bits, operating independently on each 64-bit word.
- $X \lll n$: left-rotation (circular shift) by n bits, operating independently on each 64-bit word.
- $Truncate_{[i]}(S)$: Truncation to the first i bits of bit string S .
- $Truncate'_{[i]}(S)$: Truncation to the last i bits of bit string S .

1.4 State and State Array Conversion

The state S of CHIME is a 1536-bit string, with bits indexed from 0 to 1535. S is represented as:

$$S = S[0] \parallel S[1] \parallel \dots \parallel S[1534] \parallel S[1535].$$

The 1536-bit state S can be represented as six 256-bit state arrays, denoted as $A_0, A_1, B_0, B_1, C_0, C_1$, where:

$$\begin{aligned} A_0 &= S[0] \parallel S[1] \parallel \dots \parallel S[255] \\ A_1 &= S[256] \parallel S[257] \parallel \dots \parallel S[511] \\ B_0 &= S[512] \parallel S[513] \parallel \dots \parallel S[767] \\ B_1 &= S[768] \parallel S[769] \parallel \dots \parallel S[1023] \\ C_0 &= S[1024] \parallel S[1025] \parallel \dots \parallel S[1279] \\ C_1 &= S[1280] \parallel S[1281] \parallel \dots \parallel S[1535] \end{aligned}$$

The arrangement of $A_0, A_1, B_0, B_1, C_0, C_1$ is shown in Figure 1.

A_0	A_1
B_0	B_1
C_0	C_1

Figure 1: Arrangement of State Arrays

1.5 The CHIME Hash Function

CHIME first pads the input message M to obtain M' , then divides M' into blocks, absorbs them, and outputs the hash value. The hash value lengths are 512 bits and 1024 bits, respectively. The CHIME hash function is denoted as CHIME-512 and CHIME-1024 based on the hash length. The state size of the permutation P is 1536 bits. The maximum message length supported by CHIME is at least $2^{64} - 1$ bits.

1.5.1 CHIME-512 Description

CHIME-512 adopts the Sponge construction [2], as shown in Figure 2. The state S is 1536 bits, the message block length r is 512 bits, the capacity c is 1024 bits, and the hash value length is 512 bits. The permutation P of CHIME-512 consists of 15 rounds of the round function RP .

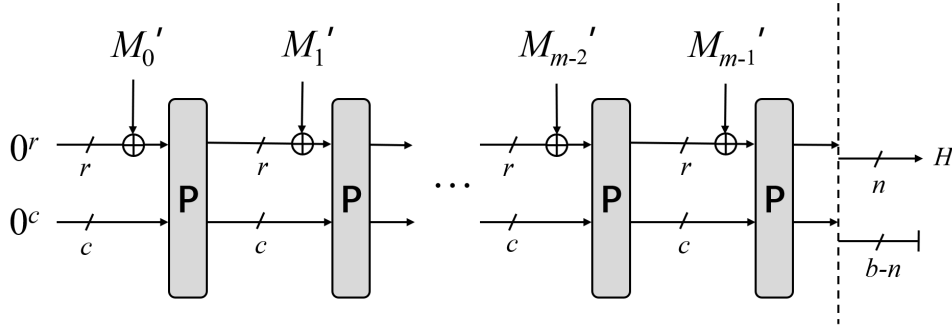


Figure 2: Sponge Construction [2]

Padding Pad the message M so that its total length is divisible by $r = 512$. First, append 1 bit “1” to the end of the message, followed by k bits “0”, and finally append 1 bit “1” to obtain the padded message M' . k is the smallest non-negative integer satisfying $(|M| + 2 + k) \equiv 0 \pmod{512}$. After padding, M' is split into m message blocks of length $r = 512$ bits: $M'_0, M'_1, \dots, M'_{m-1}$.

Initialization Initialize a 1536-bit internal state S_0 by setting all of its bits to 0.

Absorbing For each 512-bit message block M'_i ($i = 0, \dots, m-1$), XOR it with the first 512 bits of the current state, and then execute permutation P on the entire 1536-bit state:

$$S_{i+1} = P(S_i \oplus (M'_i \parallel 0^c)).$$

The absorbing phase ends after all m message blocks have been processed.

Squeezing The state after the absorbing phase is denoted as S_{final} . Truncate the first 512 bits of S_{final} as the final hash value H , i.e.:

$$H = \text{Truncate}_{512}(S_{final}).$$

1.5.2 CHIME-1024 Description

CHIME-1024 adopts the Sponge-with-feeding-forward construction [5], as shown in Figure 3. The state S is 1536 bits, the message block length r is 448 bits, the capacity c is 1088 bits, and the hash value is 1024 bits. The permutation P consists of 20 rounds of the round function RP .

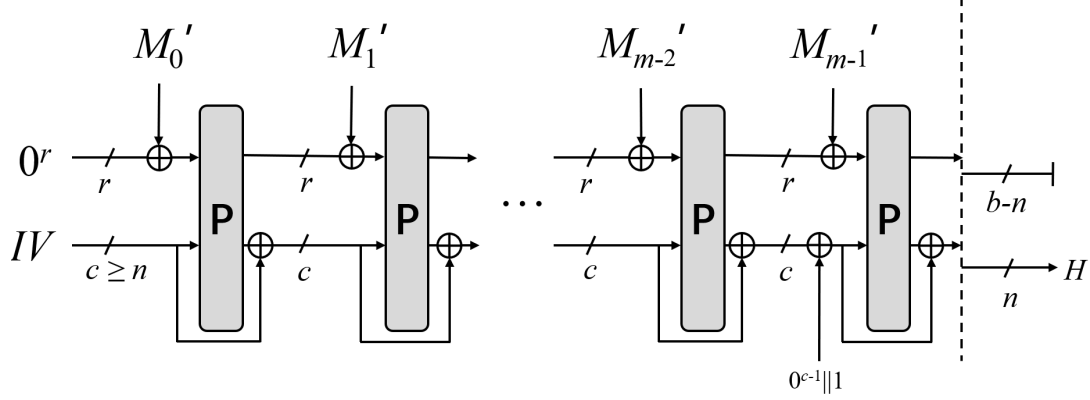


Figure 3: Sponge-with-feeding-forward Construction [5]

Padding Pad the message M so that its total length is divisible by 448. First, append 1 bit “1” to the end of the message, followed by k bits “0”, and finally append 1 bit “1” to obtain the padded message M' . k is the smallest non-negative integer satisfying $(|M| + 2 + k) \equiv 0 \pmod{448}$. After padding, M' is split into m message blocks of length 448 bits: $M'_0, M'_1, \dots, M'_{m-1}$.

Initialization Initialize a 1536-bit internal state S_0 by setting all of its bits to 0.

Absorbing and Feeding-Forward For each 448-bit message block M'_i ($i = 0, \dots, m - 1$), XOR it with the first 448 bits of the current state, and then execute the following operation on the entire 1536-bit state:

$$S_{i+1} = P(S_i \oplus (M'_i \parallel 0^c)) \oplus (0^r \parallel \text{Truncate}'_{1088}(S_i)).$$

After absorbing the $(m - 1)$ -th message block, execute $S_{m-1} \leftarrow S_{m-1} \oplus (0^{c-1} \parallel 1)$. The absorbing and feeding-forward phase ends after all m message blocks have been processed.

Squeezing The state after the absorbing phase is denoted as S_{final} . Truncate the last 1024 bits of S_{final} as the final hash value H , i.e.:

$$H = \text{Truncate}'_{1024}(S_{final}).$$

1.6 Round Function

The round function is denoted as RP , and each round can be represented as:

$$RP = \tau \cdot \chi \cdot AC \cdot \lambda.$$

The specific operations are as follows.

1.6.1 Linear Operation λ

Input: State arrays $A_0, A_1, B_0, B_1, C_0, C_1$

Output: State arrays $A_0, A_1, B_0, B_1, C_0, C_1$

$$\begin{aligned}
B_1 &\leftarrow B_1 \oplus \Pi_{perm32}(C_1) \\
B_0 &\leftarrow B_0 \oplus A_1 \\
C_0 &\leftarrow C_0 \oplus (A_0 \lll 29) \\
B_1 &\leftarrow B_1 \oplus (A_1 \lll 31) \\
C_1 &\leftarrow C_1 \oplus (A_0 \lll 13) \\
C_0 &\leftarrow C_0 \oplus B_1 \\
A_0 &\leftarrow \Pi_{perm64}(A_0) \\
C_1 &\leftarrow C_1 \oplus \Pi_{perm32}(B_1) \\
A_1 &\leftarrow \Pi_{swap128}(A_1) \\
A_0 &\leftarrow A_0 \oplus \Pi_{swap64}(C_0) \\
A_1 &\leftarrow A_1 \oplus \Pi_{perm32}(B_0) \\
B_1 &\leftarrow B_1 \oplus (A_0 \ll 7) \\
C_1 &\leftarrow C_1 \oplus (A_1 \gg 5)
\end{aligned}$$

- $\Pi_{perm32}(X)$:

$$d_1 \parallel a_1 \parallel b_1 \parallel c_1 \parallel d_0 \parallel a_0 \parallel b_0 \parallel c_0 \longleftarrow a_1 \parallel b_1 \parallel c_1 \parallel d_1 \parallel a_0 \parallel b_0 \parallel c_0 \parallel d_0$$

where $X = a_1 \parallel b_1 \parallel c_1 \parallel d_1 \parallel a_0 \parallel b_0 \parallel c_0 \parallel d_0$, and a_i, b_i, c_i, d_i ($i \in \{0, 1\}$) are 32-bit words.

- $\Pi_{perm64}(X)$:

$$b \parallel c \parallel d \parallel a \longleftarrow a \parallel b \parallel c \parallel d$$

where $X = a \parallel b \parallel c \parallel d$, and a, b, c, d are 64-bit words.

- $\Pi_{swap64}(X)$:

$$b \parallel a \parallel d \parallel c \longleftarrow a \parallel b \parallel c \parallel d$$

where $X = a \parallel b \parallel c \parallel d$, and a, b, c, d are 64-bit words.

- $\Pi_{swap128}(X)$:

$$b \parallel a \longleftarrow a \parallel b$$

where $X = a \parallel b$, and a, b are 128-bit data blocks.

1.6.2 AddRoundConstant (AC)

Input: State array A_0 , round r , round constant RC_i (see Table 7).

Output: State array A_0

$$A_0 \leftarrow A_0 \oplus RC_i$$

1.6.3 Non-linear Operation χ

Input: State arrays $A_0, A_1, B_0, B_1, C_0, C_1$, round r

Output: State arrays $A'_0, A'_1, B'_0, B'_1, C'_0, C'_1$

The non-linear layer is described by a 3-bit S-box. The S-box can be implemented in a bit-sliced format, using bitwise logical instructions to operate in parallel on 256-bit vector registers.

Let the 3-bit input variable be $x = x_0 \parallel x_1 \parallel x_2$, where x_0 is the most significant bit and x_2 is the least significant bit. The mapping rules for this 3-bit S-box are shown in Table 1.

Table 1: 3-bit S-box of CHIME

x	0	1	2	3	4	5	6	7
$\mathcal{S}(x)$	0	5	3	2	6	1	4	7

If r is an even round, independent χ operations are executed on the two lanes (A_0, B_0, C_0) and (A_1, B_1, C_1) . That is, values on the same bit positions of (A_0, B_0, C_0) or (A_1, B_1, C_1) are treated as (x_0, x_1, x_2) , input to the S-box:

$$\begin{aligned} A'_0 &= A_0 \oplus (\sim B_0 \wedge C_0) \\ B'_0 &= B_0 \oplus (\sim C_0 \wedge A_0) \\ C'_0 &= C_0 \oplus (\sim A_0 \wedge B_0) \end{aligned}$$

$$\begin{aligned} A'_1 &= A_1 \oplus (\sim B_1 \wedge C_1) \\ B'_1 &= B_1 \oplus (\sim C_1 \wedge A_1) \\ C'_1 &= C_1 \oplus (\sim A_1 \wedge B_1) \end{aligned}$$

If r is an odd round, χ is performed as follows:

$$\begin{aligned} A'_0 &= A_0 \oplus (\sim B_1 \wedge C_1) \\ B'_1 &= B_1 \oplus (\sim C_1 \wedge A_0) \\ C'_1 &= C_1 \oplus (\sim A_0 \wedge B_1) \end{aligned}$$

$$\begin{aligned} A'_1 &= A_1 \oplus (\sim B_0 \wedge C_0) \\ B'_0 &= B_0 \oplus (\sim C_0 \wedge A_1) \\ C'_0 &= C_0 \oplus (\sim A_1 \wedge B_0) \end{aligned}$$

1.6.4 Linear Operation τ

Input: State arrays $A_0, A_1, B_0, B_1, C_0, C_1$

Output: State arrays $A'_0, A'_1, B'_0, B'_1, C'_0, C'_1$

In rounds $3k, 3k+1, 3k+2$ (where k is a non-negative even integer), the state arrays are updated as follows:

$$\begin{aligned} A'_0 &= C_1, & C'_1 &= C_0, & C'_0 &= B_1 \\ B'_1 &= B_0, & B'_0 &= A_1, & A'_1 &= A_0 \end{aligned}$$

In rounds $3k, 3k+1, 3k+2$ (where k is a positive odd integer), the state arrays are updated as follows:

$$\begin{aligned} A'_0 &= A_1, & A'_1 &= B_0, & B'_0 &= B_1 \\ B'_1 &= C_0, & C'_0 &= C_1, & C'_1 &= A_0 \end{aligned}$$

2 Performance

This section provides the performances of CHIME-512, CHIME-1024, and SHA2-512.

2.1 Software Performance

Table 2 shows the software speed of CHIME and SHA2-512. We implement them in C code. We test the speed on the Intel(R) Core(TM) i7-12700F 2.10GHz processor, 64GB RAM and a 64-bit Windows 11 operating system.

Table 2: Software Speed of CHIME and SHA2-512 (Mbps)

Hash Function	128B	1KB	16KB	100MB	1GB
CHIME-512	5379.808	7656.591	8146.979	8106.658	8123.119
CHIME-1024	3522.831	4567.534	4711.410	4674.571	4659.169
SHA2-512	411.967	1970.631	3863.974	4111.810	4073.325
SHA2-512_openssl	3232.341	6485.655	7494.270	7480.632	7463.372

As seen from Table 2, the software execution speed of CHIME-512 is approximately twice that of the baseline implementation of SHA2-512, and slightly higher than the OpenSSL-based implementation of SHA2-512.

2.2 Hardware Performance

The hardware implementation for CHIME-512, CHIME-1024, and SHA2-512 are shown in Table 3 and Table 4. We implement them on the 28nm Artix-7 series xc7a35tcpg236-1 device. All hash functions implement the message padding circuits. The hash functions are written in System Verilog. The simulation, synthesis, and implementation tools used are Vivado 2024.1. Due to the limited IO interfaces of the device, the core modules of the hash functions cannot be implemented directly, so a wrapper layer is added for serial IO. All data in this section corresponds to the core module of the hash functions.

Table 3: Throughput of SHA2-512-core, CHIME-512-core, and CHIME-1024-core (Mbps)

Hash Function	128B	1KB	16KB	100MB	1GB
SHA2-512-core	230.226	549.466	674.779	685.196	685.197
CHIME-512-core	2981.251	4150.887	4381.088	4397.343	4397.345
CHIME-1024-core	2273.444	2903.895	3018.846	3023.679	3023.681

Table 4: Metrics of SHA2-512-core, CHIME-512-core, and CHIME-1024-core

Metric	SHA2-512-core	CHIME-512-core	CHIME-1024-core
Critical Path (ns)	16.967	5.552	5.652
Area (slices)	1552	2114	2509
Throughput/Area (Mbps/Slice)	0.441	2.080	1.205
Power (W)	0.083	1.325	1.186
Energy (10^{-12} J/bit)	115.521	287.395	374.066

From Table 3 and Table 4, we can get that the hardware execution speed of CHIME-512-core is 6.41 to 12.9 times that of SHA2-512-core, with a hardware area 1.36 times larger. The hardware execution speed of CHIME-1024-core is 4.41 to 9.87 times that of SHA2-512-core, with a hardware area 1.61 times larger.

By adjusting the synthesis and implementation strategies, as well as clock constraints, the tools can be guided to optimize for area or critical path. The hardware implementation parameters of CHIME-512-core and CHIME-1024-core under different strategies are shown in Table 5.

Table 5: Area, Critical Path, and Throughput of CHIME-512-core and CHIME-1024-core under Different Optimization Strategies

Hash Function	Strategy	Area (Slices)	Critical Path (ns)	Throughput (Mbps)				
				128B	1KB	16KB	100MB	1GB
CHIME-512-core	Normal Impl.	2075	15.047	1100.01	1531.58	1616.52	1622.52	1622.52
	Area Opt. 1	1757	15.386	1075.78	1497.84	1580.90	1586.77	1586.77
	Area Opt. 2	1781	21.105	784.26	1091.96	1152.51	1156.79	1156.79
	Area Opt. 3	1748	21.035	786.87	1095.59	1156.35	1160.64	1160.64
	Throughput Opt.	2128	5.395	3068.01	4271.68	4508.58	4525.31	4525.31
CHIME-1024-core	Normal Impl.	2425	14.388	893.07	1140.73	1185.89	1187.78	1187.78
	Area Opt. 1	2216	20.666	621.77	794.19	825.63	826.95	826.95
	Area Opt. 2	2233	17.610	729.67	932.02	968.91	970.46	970.46
	Area Opt. 3	2268	18.083	710.58	907.64	943.57	945.08	945.08
	Throughput Opt.	2877	5.647	2275.46	2906.47	3021.52	3026.36	3026.36

3 Choice of Parameters for CHIME

3.1 Choice of the Shift and rotation offsets in Round Function

The shift and rotation offsets in the round function are selected in terms of the avalanche effect tested over 3-6 rounds permutation. The avalanche effect from 3 rounds to 6 rounds are shown in Table 6. The minimum flip proportion represents the smallest proportion of output bit flips among the tested samples. The average flip proportion represents the average proportion across all tested samples.

Table 6: Avalanche Effect Test Results for 3-6 Rounds Permutation				
Metric	3 Rounds	4 Rounds	5 Rounds	6 Rounds
Min Flip Proportion (%)	4.9	20.4	38.9	45.1
Avg Flip Proportion (%)	21.9	42.8	49.6	50.0

3.2 Choice of the Round Constants

The round constant RC_i consists of 4 identical 64-bit words, each denoted as X_i . The 64-bit word X_i for the i -th round is generated by the following formula:

$$X_i = 0xDEADBEEF00000000 \oplus i \oplus (0x9E3779B97F4A7C15 \times (i + 1)).$$

Here, $0x9E3779B97F4A7C15$ is an unsigned 64-bit constant derived from the golden ratio. The lower 64 bits of the multiplication result are then used in the XOR operation. The round constants are shown in Table 7.

3.3 Choice of the Number of Rounds

The recommended number of rounds for CHIME-512 is 15, and the recommended number of rounds for CHIME-1024 is 20. This selection relies on the round margins established in the security analysis.

Table 7: Round Constants (Hexadecimal)

Round i	Constant X_i	Round i	Constant X_i
0	0x409AC7567F4A7C15	10	0x12CF8417783354ED
1	0xE2C34D9DFE94F82B	11	0xB4340A5EF77DD0F7
2	0x040BD3C37DDF743D	12	0xD67C908476C84D1D
3	0xA670580AFD29F057	13	0x79A516CBF612C92B
4	0xC9B8DE707C746C6D	14	0x9BED9F31755D4535
5	0x6BE164B7FBBEE87B	15	0x3DDA2578F4A7C15F
6	0x8D29EAFD7B096495	16	0x5F02ABBE73F23D75
7	0x2F167324FA53E0AF	17	0xC14B31E5F33CB96B
8	0x515EF96A799E5CB5	18	0x60B3B62B7287359D
9	0xF0877FD1F8E8D8DB	19	0x82F83C92F1D1B1B7

4 Security Claims

Let the hash value be n bits, the rate be r bits, the capacity be c bits, the squeezing rate be r' bits, and the permutation of the hash function be a random permutation. Hash functions adopting the Sponge construction [2] and the Sponge-with-feeding-forward construction [5] possess the following properties.

Property 1: A hash function adopting the Sponge construction [2] has the following generic security properties.

- The time complexity for collision attacks is $\min\{2^{n/2}, 2^{c/2}\}$.
- The time complexity for preimage attacks is $\min\{2^{\min\{n,b\}}, \max\{2^{\min\{n,b\}-r'}, 2^{c/2}\}\}$.
- The time complexity for second-preimage attacks is $\min\{2^n, 2^{c/2}\}$.

Property 2: A hash function adopting the Sponge-with-feeding-forward construction [5] has the following generic security properties.

- The time complexity for collision attacks is $2^{n/2}$.
- The time complexity for preimage attacks is 2^n .
- The time complexity for second-preimage attacks is $\min\{2^n, 2^{c-64}\}$.

Based on Properties 1 and 2, the security claims for hash functions CHIME-512 and CHIME-1024 are shown in Table 8.

Table 8: Security Claims for Hash Functions

Hash Function	Collision Attack	Preimage Attack	Second-Preimage Attack
CHIME-512	2^{256}	2^{512}	2^{512}
CHIME-1024	2^{512}	2^{1024}	2^{1024}

5 Security Analysis

This section provides a security analysis of the hash function CHIME against collision attacks, preimage attacks, and second-preimage attacks under both classical and quantum computing settings, as well as their resistance to length extension attacks.

5.1 Differential Distribution Table

Table 9 presents the Differential Distribution Table (DDT) for the S-box of CHIME, where the maximum probability for an input difference transitioning to an output difference is 2^{-2} .

Table 9: DDT of the CHIME S-box

	0	1	2	3	4	5	6	7
0	8	0	0	0	0	0	0	0
1	0	2	0	2	0	2	0	2
2	0	0	2	2	0	0	2	2
3	0	2	2	0	0	2	2	0
4	0	0	0	0	2	2	2	2
5	0	2	0	2	2	0	2	0
6	0	0	2	2	2	2	0	0
7	0	2	2	0	2	0	0	2

5.2 Collision Attacks

5.2.1 Collision Attacks on CHIME-512

This paper uses automatic tool based on SAT to get the number of active S-boxes in differential characteristics of CHIME-512. The probability of the input and output differences holding for an active S-box is 2^{-2} . It can be concluded that 15-round CHIME-512 can resist collision attacks.

- The input difference is unconstrained and the output difference in the hash value is 0, then the number of active S-boxes are as follows.
 - The number of active S-boxes for 3-round CHIME-512 is ≤ 8 .
 - The number of active S-boxes for 4-round CHIME-512 is ≤ 14 .
 - The number of active S-boxes for 5-round CHIME-512 is ≤ 51 .
 - The number of active S-boxes for 6-round CHIME-512 is ≤ 28 .
 - The number of active S-boxes for 7-round CHIME-512 is ≤ 93 .
 - The number of active S-boxes for 8-round CHIME-512 is ≤ 72 .
 - The number of active S-boxes for 9-round CHIME-512 is ≤ 58 .
 - The number of active S-boxes for 10-round CHIME-512 is ≥ 128 .

5.2.2 Collision Attacks on CHIME-1024

This paper uses automatic tool based on SAT to get the number of active S-boxes in differential characteristics of CHIME-1024. The probability of the input and output differences holding for an active S-box is 2^{-2} . It can be concluded that 20-round CHIME-1024 can resist collision attacks.

- The input difference is unconstrained and the output difference in the hash value is 0, considering the feeding-forward, the numbers of active S-boxes are as follows.
 - * The number of active S-boxes for 5-round CHIME-1024 is ≤ 34 .
 - * The number of active S-boxes for 6-round CHIME-1024 is ≤ 30 .
 - * The number of active S-boxes for 7-round CHIME-1024 is ≤ 36 .
 - * The number of active S-boxes for 8-round CHIME-1024 is ≤ 40 .
 - * The number of active S-boxes for 9-round CHIME-1024 is ≤ 97 .
 - * The number of active S-boxes for 10-round CHIME-1024 is ≤ 50 .
 - * The number of active S-boxes for 11-round CHIME-1024 is ≥ 256 .

5.3 Preimage Attacks

Based on the Meet-in-the-Middle (MitM) preimage attack in [7, 8], preimage attacks against 3-round CHIME-512 and 3-round CHIME-1024 can be constructed. This paper did not find preimage attacks on 4-round CHIME-512 or 4-round CHIME-1024.

5.3.1 State Bit Encoding

[7, 8] divides the input into two sets of independent variables and one set of constants. The two sets of independent variables are represented by red and blue variables, respectively. The constants are represented by gray variables. On the one hand, as the state bits propagate through the round functions, the output bits will be different state types. On the other hand, some relation can be deduced backwards from the hash value. Thus, matching conditions can be constructed to carry out MitM preimage attacks.

By using the automatic tool based on SAT, each state bit can be encoded using a triplet (w_0, w_1, w_2) . The meanings of the various state bit types are detailed in Table 10. Furthermore, in the cancellation state, c_{lin} and c_{non} are introduced to represent linear cancellations and non-linear cancellations, respectively.

Table 10: State Bit Encoding

Encoding (w_0, w_1, w_2)	CHIME	Description
(0, 1, 0)	gray	Gray represents constant bits
(0, 1, 1)	red	Red bits
(1, 1, 0)	blue	Blue bits
(0, 0, 0)	white	White represents non-linear parts where red and blue variables are multiplied
(0, 0, 1)	pink	Pink represents non-linear parts with red variables multiplication but no blue variable involvement
(1, 1, 1)	green	Green represents linear combination of red and blue variables
(1, 0, 1)	maroon	Maroon represents non-linear part of red variables multiplication in addition with linear part of blue variables

5.3.2 Complexity Calculation for Preimage Attacks

According to the conclusions in [8], the time complexity and memory complexity for the preimage attack on CHIME-512 are calculated as follows:

$$T_{512} = \max \left\{ 2^{512-\sigma+n_c}, 2^{h-\min(n_{\text{blue}}, n_{\text{red}}, n_{\text{matching}})} \right\},$$

$$M_{512} = \max \left\{ 2^{n_{\text{red}}-n_{\text{lin}}}, 2^{n_{\text{blue}}} \right\}.$$

The time complexity and memory complexity for the preimage attack on CHIME-1024 are calculated as follows:

$$T_{1024} = \max \left\{ 2^{1024-\min(n_{\text{blue}}, n_{\text{red}}-\sigma, n_{\text{matching}})}, 2^{\sigma+n_c+576} \right\},$$

$$M_{1024} = \max \left\{ 2^{n_{\text{red}}-n_{\text{lin}}}, 2^{n_{\text{blue}}} \right\}.$$

Here $\sigma = n_{\text{lin}} + n_{\text{non}}$ denotes the total number of cancellations, n_c denotes the number of bit conditions that the capacity part of the initial state must satisfy, n_{matching} denotes the number of matching points. n_{red} and n_{blue} represent the number of red bits and blue bits, respectively.

For CHIME-512, using the end-matching scheme (see Figure 4), we introduce red and blue variables from the initial state and propagate them forward to the χ operation of the 2nd or 3rd round (rounds are indexed starting from 0), matching with the hash value, as illustrated in Figure 4.

Due to the characteristics of the round function, when the red and blue variables propagate to the 2nd round, a portion of the output capacity must be guessed to potentially achieve a MitM match. Additionally, because the round function of CHIME differs from SHA-3, the linearization of round 0 in CHIME cannot directly adopt the linearization method of SHA-3 [7, 8]. This paper employs automatic tool based on SAT to search the linearization path with the maximum degrees of freedom. Therefore, n_c varies across different linearization paths.

For CHIME-1024 with a feeding-forward mechanism, this paper utilizes the end-matching scheme (see Figure 5(a)) and feeding-forward mid-matching scheme (see Figure 5(b)) to give preimage attacks on round-reduced CHIME-1024. The latter (see Figure 5(b)) introduces red and blue variables from the output of round 0, propagates them backward to the initial state to obtain conditions on the capacity, i.e., n_c non-constant bits appearing in the capacity. It then propagates through the feeding-forward state to the end of round 3, and continues backward for one round. Then it matches the bit relations from the forward propagation at the output of χ in round 2.

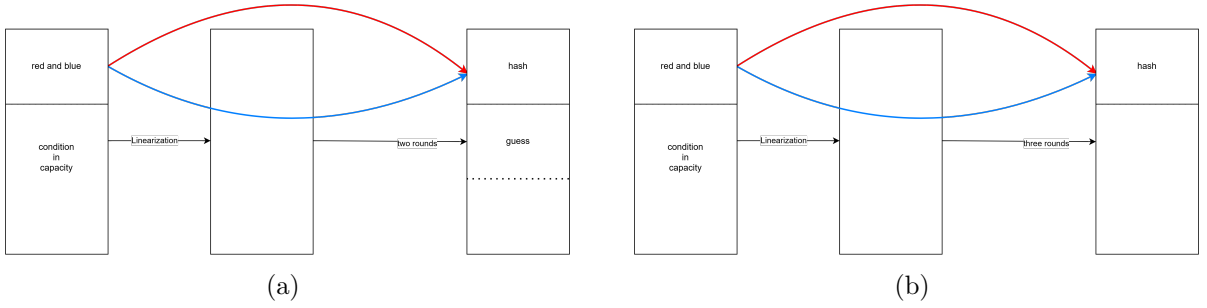


Figure 4: 3-Round End-Matching and 4-Round End-Matching for CHIME-512

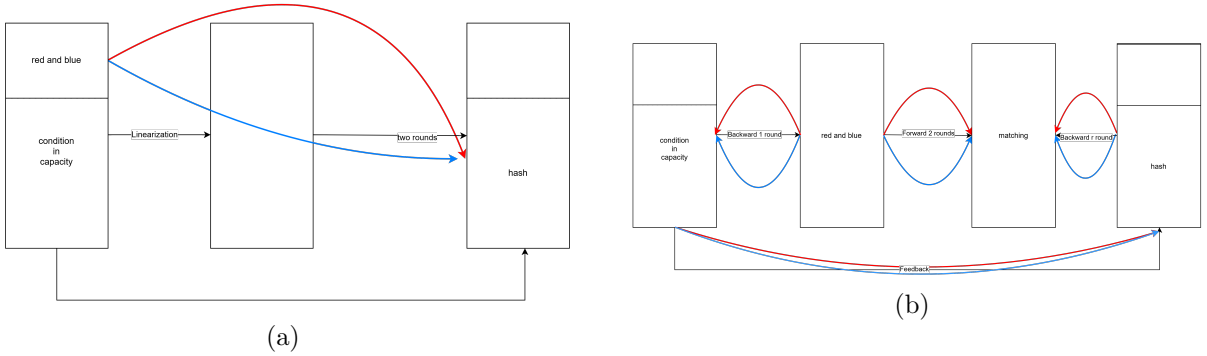


Figure 5: 3-Round End-Matching and 4-Round Mid-Matching for CHIME-1024

5.3.3 Preimage Attack Results

The preimage analysis results for CHIME-512 and CHIME-1024 are shown in Table 11 and Table 12, respectively, where T represents time complexity and M represents memory complexity. The 3-round MitM paths for CHIME-512 and CHIME-1024 are shown in Figure 6 and Figure 7, respectively. Compared with SHA-3, it is more difficult to linearize CHIME. And the remaining degrees of freedom in the system after linearization are fewer than that of SHA-3.

3-round CHIME-512 and CHIME-1024 exhibit high resistance to MitM preimage attacks. This paper did not find preimage attacks for 4-round CHIME-512 or CHIME-1024.

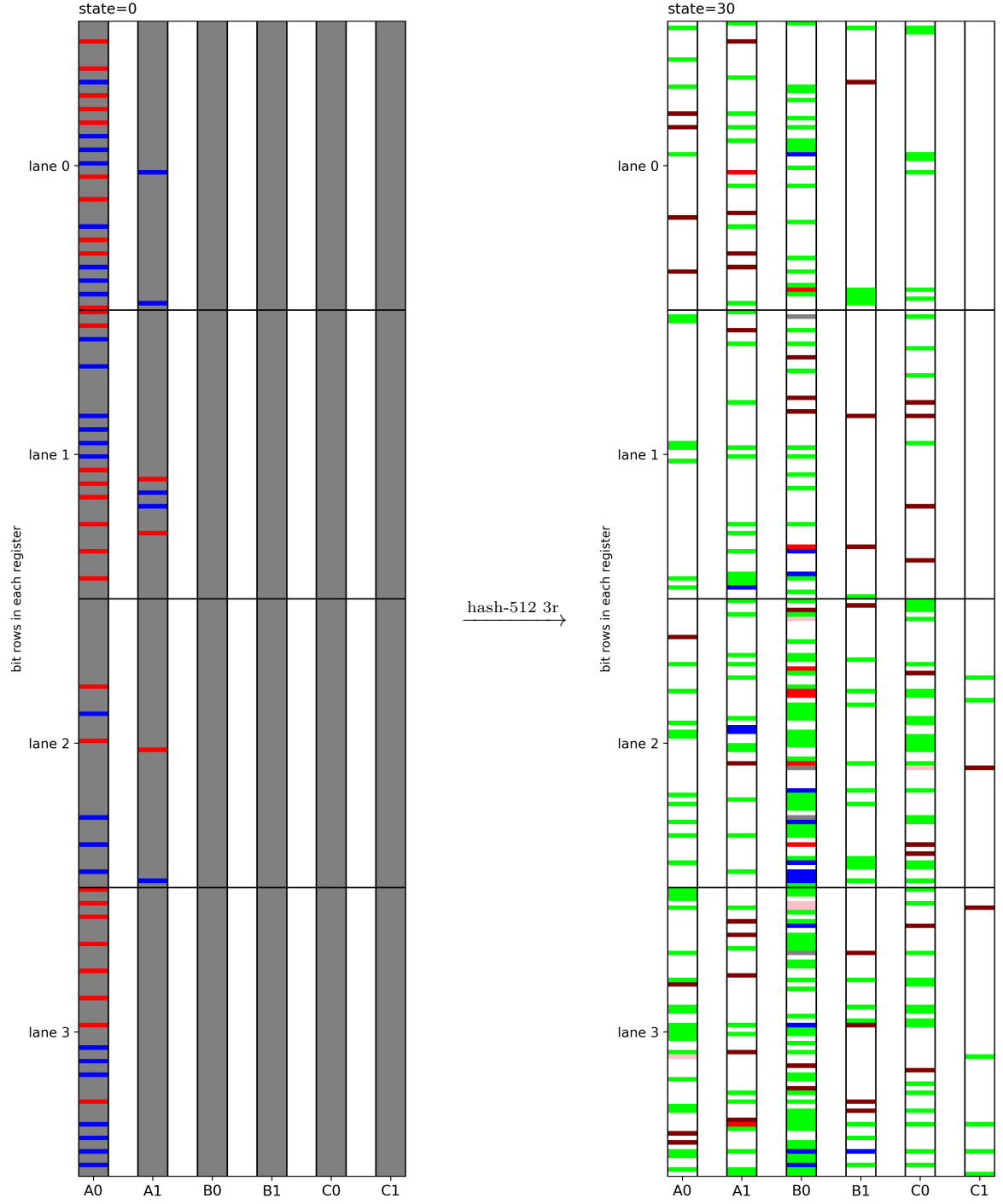


Figure 6: MitM End-Matching Path of 3-Round CHIME-512

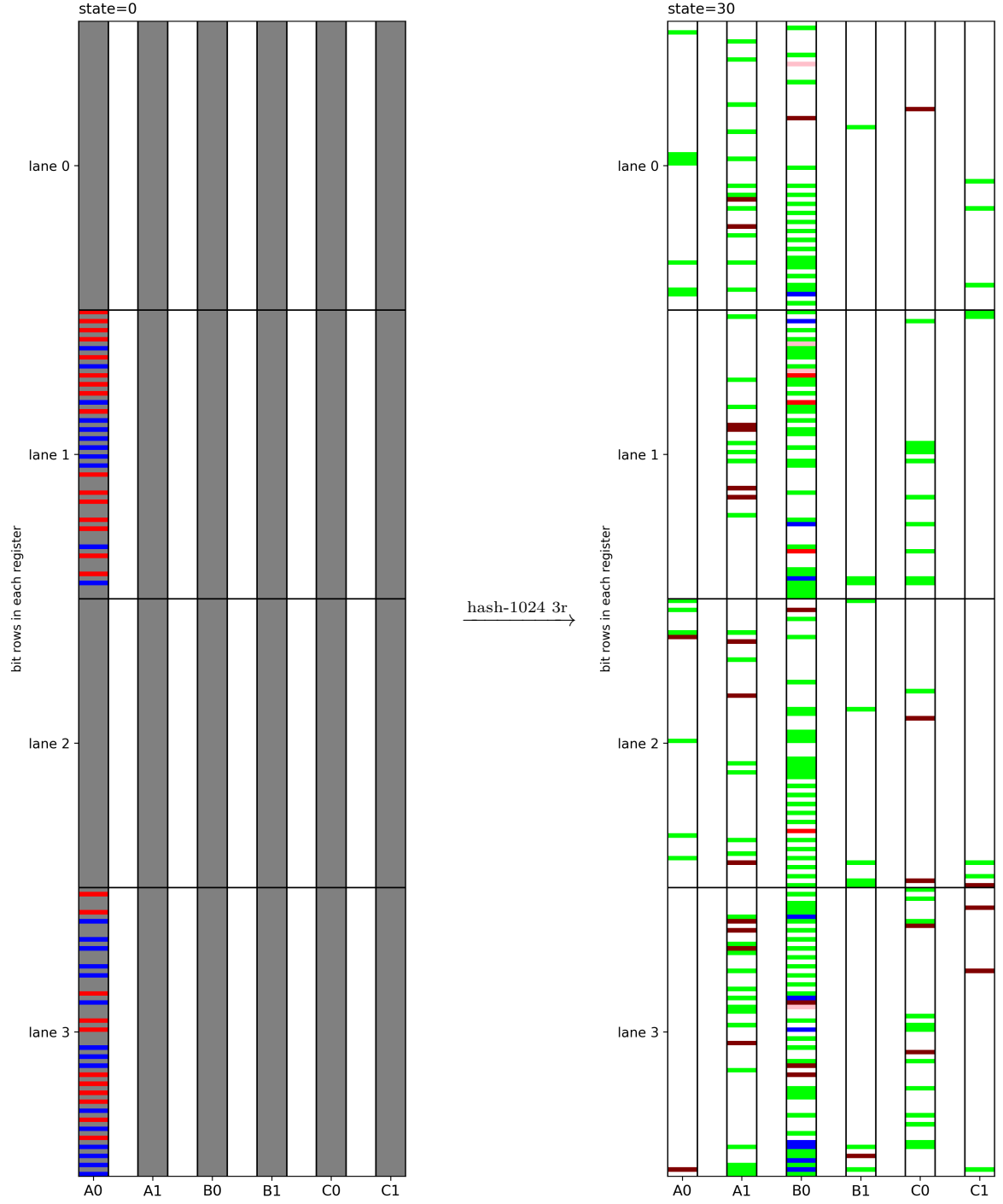


Figure 7: MitM End-Matching Path of 3-Round CHIME-1024

Table 11: Preimage Attack Results on CHIME-512

Rounds	n_{red}	n_{blue}	n_c	n_{lin}	n_{non}	σ	n_{matching}	T	M
3 Rounds	31	30	480	1	0	1	30	2^{482}	2^{30}
4 Rounds	49	8	450	81	0	81	0	2^{544}	2^8

Table 12: Preimage Attack Results on CHIME-1024

Scheme	n_{red}	n_{blue}	n_c	n_{lin}	n_{non}	σ_{small}	n_{matching}	T	M
End-Matching 3R	27	26	415	1	0	1	182	2^{998}	2^{26}
End-Matching 4R	52	1	415	20	0	20	0	2^{1024}	2^{32}
Feedback Mid-Matching 4R	390	58	1037	56	0	56	0	2^{1669}	2^{372}

5.4 Second-Preimage Attack

The results of second-preimage attacks for CHIME-512 and CHIME-1024 are consistent with that of the preimage attacks. The 15-round CHIME-512 and 20-round CHIME-1024 can resist second-preimage attacks.

5.5 Quantum Attack

The complexity for Grover’s algorithm [4] to find a hash function collision is $T = O(2^{n/2})$, $S = O(1)$ or $T = O(2^{n/3})$, $S = O(2^{n/3})$. Here the time complexity T corresponds to the depth of the quantum circuit under the quantum circuit model, and the space complexity S corresponds to the scale of the quantum circuit. Currently, there is no generic quantum collision attack satisfying $T \times S < 2^{n/2}$. If a collision attack satisfying $T \times S < 2^{n/2}$ can be found for a specific hash function, it is considered an effective quantum collision attack under the quantum time-space tradeoff.

The Sponge construction [2] and the Sponge-with-feeding-forward construction [5] possess no exploitable structural weaknesses [3]. Thus, the bound against quantum attacks is the generic bound [4].

5.6 Resistance to Length Extension Attacks

Length extension attacks occur when an attacker, ignorant of an input M but knowing its hash value $H(M)$, attempts to calculate the value of $H(M \parallel X)$ for some string X . For a hash function based on the Sponge construction, a length extension attack is successful only if one can recover the internal state at the end of the squeezing phase of M , which fundamentally reduces to a state recovery problem [2]. If a hash function is secure against state recovery attacks, then it can resist length extension attacks.

For the Sponge-with-feeding-forward construction [5], it XORs a constant $0^{c-1} \parallel 1$ into the capacity part before absorbing the final message block. Therefore, it can resist length extension attack.

In conclusion, both CHIME-512 and CHIME-1024 can resist length extension attacks.

5.7 Linear Analysis

Table 13 presents the Linear Approximation Table (LAT) for the S-box, where the maximum linear bias is 2^{-2} . Based on automated modeling analysis, the linear analysis results for permutation P are as follows.

- For a 4-round permutation, the minimum number of active S-boxes in linear analysis is ≤ 15 and > 10 .
- For a 5-round permutation, the minimum number of active S-boxes in linear analysis is ≤ 31 .

Table 13: Linear Approximation Table (LAT) of CHIME

	0	1	2	3	4	5	6	7
0	8	0	0	0	0	0	0	0
1	0	4	0	4	0	-4	0	4
2	0	0	4	-4	0	0	4	4
3	0	4	-4	0	0	4	4	0
4	0	0	0	0	4	4	-4	4
5	0	-4	0	4	4	0	4	0
6	0	0	4	4	-4	4	0	0
7	0	4	4	0	4	0	0	-4

- For a 6-round permutation, the minimum number of active S-boxes in linear analysis is ≤ 45 .
- For a 7-round permutation, the minimum number of active S-boxes in linear analysis is ≤ 63 .
- For a 8-round permutation, the minimum number of active S-boxes in linear analysis is ≤ 126 .

5.8 Randomness Testing of CHIME

To evaluate the statistical properties of the hash function CHIME, the comprehensive test suite specified in NIST Special Publication 800-22 Revision 1a was utilized [6]. The randomness testing results for CHIME are provided in Appendix A. The test results indicate that sequences generated by Random Message Mode, Output Feedback Mode, Low Density Mode, and High Density Mode all pass the 15 randomness tests of NIST SP-800-22 [6]. It is concluded that there is no statistically difference between the tested sequences and ideal random sequences.

6 Feature Claims

(1) Innovativeness of the Hash Function

The efficiency of the hash function across different implementation platforms was considered in the design phase. In terms of software implementation, the state organization and round function structure of this hash function are highly conducive to parallel processing using the AVX instruction set. In terms of hardware implementation, the hash function as a whole is composed of highly regular linear transformations, non-linear transformations, and state rearrangement operations. This facilitates the construction of iterative implementations and provides the possibility for further partial unrolling optimizations.

(2) Simplicity of the Hash Function

By adopting Sponge construction and its variants, the overall structure is concise. The round function of the permutation does not rely on complex table lookup operations or large-scale conditional control logic. The round function mainly consists of XOR, (circular) shifts and non-linear transformations, making it easy to describe and implement.

(3) Flexibility of the Hash Function

The hash function allows for highly efficient and flexible implementations in both software and hardware environments.

(4) Implementation Advantages Across Different Platforms

This hash function demonstrates excellent implementation adaptability across various platforms. On general-purpose processor platforms, the round function can be conveniently mapped to the AVX instruction set, thereby enhancing the parallel processing capability of the software implementation. In hardware implementation, the data path is regular, making it suitable for constructing compact iterative implementations.

References

- [1] Institute of Commercial Cryptography Standards (ICCS), Next-Generation Commercial Cryptography (NGCC) Global Solicitation Activity. <https://www.niccs.org.cn/symbbzyjy/>
- [2] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche. Cryptographic Sponge Functions. 2011. <http://sponge.noekeon.org/CSF-0.1.pdf>
- [3] Xiaoyang Dong, Shun Li, Phuong Pham, Guoyan Zhang. Quantum Attacks on Hash Constructions with Low Quantum Random Access Memory. ASIACRYPT (3), pp. 3-33, 2023.
- [4] L.K. Grover. A fast quantum mechanical algorithm for database search. Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
- [5] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, Meiqin Wang. Permutation-Based Hash from Non-Idealized Assumptions: Adding Feeding-Forward to Sponge. <https://eprint.iacr.org/2025/1006>
- [6] National Institute of Standards and Technology. A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications. Technical Report Special Publication 800-22 Rev. 1a, National Institute of Standards and Technology, Gaithersburg, MD, 2010.
- [7] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In EUROCRYPT 2023, Proceedings, Part IV, volume 14007, pages 158-188. Springer, 2023.
- [8] Lingyue Qin, Boxin Zhao, Qingliang Hou, Keting Jia, Gaoli Wang, Xiaoyang Dong. Linear Cancellations in the MitM Attacks on Sponge Functions. IACR Trans. Symmetric Cryptol. 2025(3): 633-692, 2025.

A Appendix: Randomness Tests of CHIME

To evaluate the output properties of the hash function under various input distribution environments, we tested four message input modes with different features: Random Message Mode, Output Feedback Mode, Low Density Mode, and High Density Mode. The sequences generated by each mode were subjected to the 15 randomness tests specified in NIST SP 800-22 [6].

- 1) The Frequency (Monobit) Test
- 2) Frequency Test within a Block
- 3) The Runs Test
- 4) Tests for the Longest-Run-of-Ones in a Block
- 5) The Binary Matrix Rank Test
- 6) The Discrete Fourier Transform (Spectral) Test
- 7) The Non-overlapping Template Matching Test
- 8) The Overlapping Template Matching Test
- 9) Maurer’s “Universal Statistical” Test
- 10) The Linear Complexity Test
- 11) The Serial Test
- 12) The Approximate Entropy Test
- 13) The Cumulative Sums (Cusums) Test
- 14) The Random Excursions Test
- 15) The Random Excursions Variant Test

The specific testing purposes, methods, principles, and decision criteria are detailed in NIST SP 800-22 document [6]. We utilize the following methods to test randomness.

Suppose there are m sequences under test, each of length l . The 15 tests listed above are applied to each sequence. For each test, m P -values and the proportion of sequences passing the test are generated. Based on these data, the following two methods are used to evaluate randomness.

Method 1: Proportion of Passing Sequences

Statistical testing was performed on the proportion of sequences passing the test. Let r denote the proportion of sequences passing the test. Let the significance level be $\alpha = 0.01$, and $\hat{p} = 1 - \alpha$. The data is considered to pass the randomness test at a confidence level of 0.01 if $r \in \left[\hat{p} - 3\sqrt{\frac{\hat{p}(1-\hat{p})}{m}}, \hat{p} + 3\sqrt{\frac{\hat{p}(1-\hat{p})}{m}} \right]$.

Method 2: Uniformity of P -values

Test whether the P -values are uniformly distributed over the interval $[0, 1]$. When all sequences under test are random, the P -values should be uniformly distributed across $[0, 1]$. According to Pearson’s theorem, a χ^2 (chi-square) goodness-of-fit test can be applied to evaluate the randomness of the P -value distribution. The method is as follows:

Divide the interval $[0, 1]$ into 10 equal sub-intervals: $[0, 0.1)$, $[0.1, 0.2)$, $\dots$, $[0.9, 1]$. Let F_i be the number of P -values falling into the i -th sub-interval. Then $S_{obs} = \frac{\sum_{i=1}^{10} (F_i - \frac{m}{10})^2}{\frac{m}{10}}$ asymptotically follows a χ^2 distribution with 9 degrees of freedom, where m is the number of tested sequences. Therefore, if $P\text{-value} = \text{igamc}\left(\frac{9}{2}, S_{obs}\right) \geq 0.0001$, the data is considered to pass the randomness test at a confidence level of 0.01. The definition of the function igamc is provided in the NIST SP 800-22 document.

Note: In the randomness test suite, the Non-overlapping Template Matching Test comprises 148 independent templates. At the set significance level of $\alpha = 0.01$, the theoretical expected number of Type I errors (false rejections) is 1.48. Therefore, 1 to 2 failing templates is a reasonable deviation

and does not constitute valid evidence to conclude the sequence is non-random. Similarly, the theoretical expected number of Type I errors for the Random Excursions Variant Test is 0.18, so 1 failing template also falls within expected reasonable deviations.

To meet the statistical sample size requirements of the test suite, the total length of the generated sequences in this test was set to 10^9 bits, divided into 1,000 sequences of 1,000,000 bits each. CHIME-512 and CHIME-1024 were tested respectively.

A.0.1 Random Message Mode Testing

For CHIME-512 and CHIME-1024, random message sequences $\{M_i\}$ of 512 bits and 448 bits were generated respectively. Their corresponding hash values $H(M_i)$ were computed and concatenated to obtain 1,000 sequences of 1,000,000 bits. Randomness testing was performed on these sequences, and the results are shown in Table 14.

Table 14: Random Message Mode Test Results

Test Item	CHIME-512 Test Data (<i>P-value</i> / Proportion)	CHIME-512 Conclusion	CHIME-1024 Test Data (<i>P-value</i> / Proportion)	CHIME-1024 Conclusion
Frequency	0.786830 / 997/1000	Pass	0.560545 / 993/1000	Pass
Block Frequency	0.869278 / 990/1000	Pass	0.832561 / 992/1000	Pass
Cumulative Sums	≥ 0.639202 / 2/2	Pass	≥ 0.498313 / 2/2	Pass
Runs	0.976266 / 994/1000	Pass	0.363593 / 991/1000	Pass
Longest Run of Ones	0.121616 / 993/1000	Pass	0.165340 / 993/1000	Pass
Rank	0.492436 / 991/1000	Pass	0.957612 / 992/1000	Pass
FFT	0.234373 / 989/1000	Pass	0.583145 / 987/1000	Pass
Non-Overlapping Template	≥ 0.002154 / 148/148	Pass	≥ 0.006425 / 147/148	Pass(1 Item Fail)
Overlapping Template	0.148653 / 987/1000	Pass	0.518106 / 981/1000	Pass
Universal	0.554420 / 982/1000	Pass	0.004365 / 989/1000	Pass
Approximate Entropy	0.878618 / 989/1000	Pass	0.589341 / 992/1000	Pass
Random Excursions	≥ 0.082043 / 8/8	Pass	≥ 0.023835 / 8/8	Pass
Random Excursions Variant	≥ 0.043214 / 18/18	Pass	≥ 0.034234 / 18/18	Pass
Serial	≥ 0.419021 / 2/2	Pass	≥ 0.601766 / 2/2	Pass
Linear Complexity	0.119508 / 993/1000	Pass	0.448424 / 988/1000	Pass

The test results indicate that the sequences generated in random message mode passed all 15 tests, showing no significant difference between the tested sequences and true random sequences.

A.0.2 Output Feedback Mode Testing

The operation of the output feedback mode is illustrated in Figure 8:

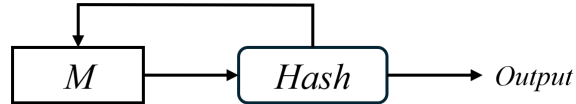


Figure 8: Output Feedback Mode

For CHIME-512 and CHIME-1024, random message sequences M_i of 512 bits and 448 bits were respectively generated. Operating in the output feedback mode, the hash value output from the previous round was used as the input for the current round. The iterations were performed, and the hash values were concatenated into a single sequence. From this, 1,000 sequences of 1,000,000 bits were obtained for testing. The randomness test results are shown in Table 15.

The test results indicate that the sequences generated in output feedback mode passed all 15 tests, showing no significant difference between the tested sequences and true random sequences.

Table 15: Output Feedback Mode Test Results

Test Item	CHIME-512 Test Data (<i>P-value</i> / Proportion)	CHIME-512 Conclusion	CHIME-1024 Test Data (<i>P-value</i> / Proportion)	CHIME-1024 Conclusion
Frequency	0.781106 / 990/1000	Pass	0.373625 / 985/1000	Pass
Block Frequency	0.875539 / 991/1000	Pass	0.597620 / 992/1000	Pass
Cumulative Sums	≥ 0.337688 / 2/2	Pass	≥ 0.130369 / 2/2	Pass
Runs	0.761719 / 996/1000	Pass	0.289667 / 985/1000	Pass
Longest Run of Ones	0.498313 / 992/1000	Pass	0.591409 / 985/1000	Pass
Rank	0.104993 / 990/1000	Pass	0.729870 / 992/1000	Pass
FFT	0.046568 / 984/1000	Pass	0.006710 / 980/1000	Pass
Non-Overlapping Template	≥ 0.001138 / 147/148	Pass(1 Item Fail)	≥ 0.000663 / 146/148	Pass(2 Items Fail)
Overlapping Template	0.786830 / 984/1000	Pass	0.068571 / 992/1000	Pass
Universal	0.012650 / 985/1000	Pass	0.246750 / 984/1000	Pass
Approximate Entropy	0.084037 / 987/1000	Pass	0.099513 / 990/1000	Pass
Random Excursions	≥ 0.011077 / 8/8	Pass	≥ 0.110595 / 8/8	Pass
Random Excursions Variant	≥ 0.067569 / 18/18	Pass	≥ 0.000997 / 17/18	Pass(1 Item Fail)
Serial	≥ 0.494392 / 2/2	Pass	≥ 0.011959 / 2/2	Pass
Linear Complexity	0.936823 / 989/1000	Pass	0.820143 / 988/1000	Pass

A.0.3 Low Density Mode Testing

For CHIME-512 and CHIME-1024, low density message sequences $\{M_i\}$ (message strings with Hamming weights of 0, 1, 2, and 3) of 512 bits and 448 bits were generated respectively for hashing. The hash values were concatenated into a single sequence, resulting in 1,000 sequences of 1,000,000 bits. Randomness testing was performed on these sequences, and the test results are shown in Table 16.

Table 16: Low Density Mode Test Results

Test Item	CHIME-512 Test Data (<i>P-value</i> / Proportion)	CHIME-512 Conclusion	CHIME-1024 Test Data (<i>P-value</i> / Proportion)	CHIME-1024 Conclusion
Frequency	0.476911 / 989/1000	Pass	0.482707 / 992/1000	Pass
Block Frequency	0.723804 / 990/1000	Pass	0.610070 / 987/1000	Pass
Cumulative Sums	≥ 0.566688 / 2/2	Pass	≥ 0.345650 / 2/2	Pass
Runs	0.715679 / 990/1000	Pass	0.848027 / 991/1000	Pass
Longest Run of Ones	0.686955 / 986/1000	Pass	0.431754 / 987/1000	Pass
Rank	0.593478 / 986/1000	Pass	0.906069 / 996/1000	Pass
FFT	0.028625 / 989/1000	Pass	0.279844 / 991/1000	Pass
Non-Overlapping Template	≥ 0.001895 / 148/148	Pass	≥ 0.003473 / 148/148	Pass
Overlapping Template	0.319084 / 987/1000	Pass	0.358641 / 992/1000	Pass
Universal	0.672470 / 986/1000	Pass	0.488534 / 989/1000	Pass
Approximate Entropy	0.821937 / 990/1000	Pass	0.203351 / 988/1000	Pass
Random Excursions	≥ 0.202617 / 8/8	Pass	≥ 0.207730 / 8/8	Pass
Random Excursions Variant	≥ 0.038695 / 18/18	Pass	≥ 0.035174 / 18/18	Pass
Serial	≥ 0.355364 / 2/2	Pass	≥ 0.429923 / 2/2	Pass
Linear Complexity	0.765632 / 988/1000	Pass	0.229559 / 990/1000	Pass

The test results indicate that the sequences generated in low density mode passed all 15 tests, showing no significant difference between the tested sequences and true random sequences.

A.0.4 High Density Mode Testing

For CHIME-512 and CHIME-1024, high density message sequences $\{M_i\}$ (message strings with Hamming weights of 512, 511, 510, 509) of 512 bits and (message strings with Hamming weights of 448, 447, 446, 445) of 448 bits were respectively generated for hashing. The hash values were concatenated into a single sequence, yielding 1,000 sequences of 1,000,000 bits. Randomness testing was performed on these sequences, and the test results are shown in Table 17.

Table 17: High Density Mode Test Results

Test Item	CHIME-512 Test Data (<i>P-value</i> / Proportion)	CHIME-512 Conclusion	CHIME-1024 Test Data (<i>P-value</i> / Proportion)	CHIME-1024 Conclusion
Frequency	0.190654 / 986/1000	Pass	0.051281 / 990/1000	Pass
Block Frequency	0.880145 / 991/1000	Pass	0.260930 / 982/1000	Pass
Cumulative Sums	≥ 0.081013 / 2/2	Pass	≥ 0.204439 / 2/2	Pass
Runs	0.670396 / 992/1000	Pass	0.248014 / 994/1000	Pass
Longest Run of Ones	0.645448 / 985/1000	Pass	0.524101 / 989/1000	Pass
Rank	0.603841 / 989/1000	Pass	0.548314 / 987/1000	Pass
FFT	0.457825 / 988/1000	Pass	0.887645 / 983/1000	Pass
Non-Overlapping Template	≥ 0.005017 / 148/148	Pass	≥ 0.004495 / 147/148	Pass(1 Item Fail)
Overlapping Template	0.948298 / 989/1000	Pass	0.282626 / 988/1000	Pass
Universal	0.373625 / 990/1000	Pass	0.570792 / 985/1000	Pass
Approximate Entropy	0.231956 / 986/1000	Pass	0.124476 / 991/1000	Pass
Random Excursions	≥ 0.006813 / 8/8	Pass	≥ 0.003577 / 8/8	Pass
Random Excursions Variant	≥ 0.003236 / 18/18	Pass	≥ 0.105618 / 18/18	Pass
Serial	≥ 0.160805 / 2/2	Pass	≥ 0.244236 / 2/2	Pass
Linear Complexity	0.221317 / 993/1000	Pass	0.047478 / 986/1000	Pass

The test results indicate that the sequences generated in high density mode passed all 15 tests, showing no significant difference between the tested sequences and true random sequences.